

ECE 5610: Sensors & Instrumentation



Analytical Balance Project

Sam Ruttgaizer - 202312489

April 7th, 2026

Executive Summary

This project presents the design, implementation, and evaluation of an automated analytical balance capable of measuring masses in the 50 g to 500 g range. The system integrates an optical sensing mechanism, a precision linear actuator, and a closed-loop control system to achieve accurate and repeatable measurements. The sensor uses an infrared emitter, dual photodiodes, and a reflective mirror to detect beam displacement, with signal conditioning via transimpedance and instrumentation amplifiers. A stepper motor-driven linear actuator adjusts a counterweight along the beam to achieve balance, while an ESP32 microcontroller executes calibration, control, and measurement functions.

A proportional-integral-derivative (PID) control strategy was implemented to ensure stable and responsive balancing, with additional logic for homing, repeatability, and disturbance handling. Calibration was performed using known weights and a piecewise linear mapping between motor steps and mass, enabling accurate conversion of mechanical position into weight readings displayed on an LCD interface.

Experimental results demonstrate that the system achieves an accuracy within approximately ± 5 g across the operating range, with good repeatability and minimal drift over time. The balance exhibits stable operation, reasonable response times (8–20 seconds depending on load), and a compact physical design. However, sensitivity to weight placement and certain mechanical limitations were identified as areas for improvement.

Overall, the project successfully demonstrates a functional, precise analytical balance that combines optical sensing, embedded control, and mechanical design, providing a strong foundation for future refinement and enhancement.

Table of Contents

Executive Summary	i
Table of Figures	iii
Table of Tables	iv
List of Acronyms.....	iv
Introduction.....	1
1.0 Overall Design and Methodology	1
1.1 Sensor Design	1
1.1.1 Sensor Conditioning.....	3
1.1.2 Sensor Validation	4
1.2 Actuator Design	6
1.2.1 CAD Design.....	8
1.2.2 Controlling the Stepper Motor	11
1.3 Control System Design	12
1.3.1 Proportional Integral Derivative	13
1.3.2 LCD Output	14
2.0 Internal System Performance.....	16
2.1 Calibration, Error, Repeatability	16
2.1.1 Calibrating the Balance.....	16
2.1.2 Error of Measurement	17
2.1.3 Repeatability of Measurement	18
2.2 Speed of Response	19
2.3 Output Stability	19
3.0 External System Performance.....	19
3.1 Sensitivity to Weight Placement and Temperature Effects	19
3.3 Compactness, Ease of Operation, Stopping Mechanism	21
3.2 Ruggedness and Long-Term Reliability	22
4.0 Problems and Improvements.....	22
4.1 Development and Testing Struggles	22
4.2 Potential Improvements	23
4.3 Used References.....	23
Conclusion	23
References.....	24

Table of Figures

Figure 1: Side view of sensor mounting; sensors placed in a tube to be positioned optimally from the balance beam; the mirror can be seen attached to the beam.	2
Figure 2: Top view of the sensor, each photodiode is positioned an equal distance from the IR emitter, and there are walls that block any IR leakage.	2
Figure 3: Functional diagram of how the sensor operates as the beam shifts its balance.	3
Figure 4: Circuit diagram of the signal conditioning of the sensor reading.	3
Figure 5: The final circuit configuration after testing.	6
Figure 6: Linear screw actuator with a NEMA 11 stepper motor, attached is the 250-gram counterweight that slides along the beam.	7
Figure 7: The beam's pivot point, assembled with a 4mm steel rod and 2 bearings attached to the base.	7
Figure 8: Balance beam that allows the sliding of the counterweight.	8
Figure 9: The stepper mount connects to the linear actuator and allows the counterweight to freely move vertically, while horizontal movement is controlled.	9
Figure 10: Collar that interacts with the stepper mount.	9
Figure 11: The base of the scale correctly positions the sensor, beam, and actuator for precise measurements.	10
Figure 12: Full system view, all 3D printed parts assembled to complete the mechanical design.	10
Figure 13: Wiring diagram of the TMC2209 to the NEMA 11 motor and to the ESP32, through the pins D2-D5.	11
Figure 14: State diagram of the overall program flow.	12
Figure 15: During the calibration state, the LCD outputs "Calibrating".	15
Figure 16: During balancing, the LCD shows the measured weight in grams, to one decimal point.	15
Figure 17: The entire powered system, the LCD outputs the weight in grams, for reference, a 200-gram scientific weight is being balanced.	15
Figure 18: The piecewise function after calibration.	17
Figure 19: Overlay of both the actual and measured lines, as well as the percentage error of each measurement.	18
Figure 20: A center mark is on the weigh pan to aid the placement of the weight in the optimal position.	20
Figure 21: An external DC barrel jack allows simple powering of the balance.	21
Figure 22: An external USB port on the other side of the base allows for reprogramming of the ESP32.	21

Table of Tables

Table 1: TIA voltage output readings, one mirror attached to the beam, directly above the IR emitter.	4
Table 2: TIA voltage output readings, two mirrors attached to the beam longitudinally, centered on the IR emitter.....	4
Table 3: TIA output voltages using a 100k Ω feedback resistor.	5
Table 4: INA output throughout the beam's travel, gain = 4 ($RG=22k\Omega$).	5
Table 5: INA output throughout the beam's travel, gain = 2 ($RG = 67k\Omega$).....	5
Table 6: INA output throughout the beam's travel, gain = 1 (gain resistor removed).....	6
Table 7: Average step value for weights from 50 to 400 in 50-gram increments.	16
Table 8: Slope and intercept of each section of the piecewise linear calibration function.	16
Table 9: A comparison between the actual weight and measured weight across a range of 10 values.	17
Table 10: A 220-gram weight measured 10 times, testing repeatability of measurements.	18
Table 11: Stability of measurement across a 10-minute time window.....	19
Table 12: A test of how placement on the weigh pan affects measurement reading	20

List of Acronyms

ADC – Analog to Digital Converter
GPIO – General Purpose Input-Output
IC – Integrated Circuit
INA – Instrumentational Amplifier
IR – Infrared
LCD – Liquid Crystal Display
PCB – Printed Circuit Board
PID – Proportional Integral Derivative
PLA – Polylactic Acid
PWM – Pulse Width Modulation
TIA – Transimpedance Amplifier
UART – Universal Asynchronous Receiver-Transmitter

Introduction

Accurate mass measurement is essential in scientific, industrial, and everyday applications, requiring systems that are both precise and reliable. Traditional analytical balances often rely on high-cost load cells or electromagnetic force restoration methods, which can be complex and expensive. This project explores an alternative approach by developing a compact, automated balance that uses optical sensing and mechanical actuation to determine mass.

The system is based on a beam balance principle, in which an unknown mass is counterbalanced by a movable counterweight. The position of this counterweight, when equilibrium is reached, is directly related to the measured mass. To detect the balance point, an optical sensor composed of an infrared emitter, photodiodes, and a reflective mirror is used. The resulting signal is conditioned and interpreted by a microcontroller, which controls a stepper motor to adjust the counterweight position.

The design integrates three main subsystems: the sensor and its signal conditioning circuitry, the actuator and mechanical structure, and the embedded control system. A proportional-integral-derivative control algorithm is implemented to ensure stable, precise balancing, while calibration techniques map actuator position to mass.

This report details the full design process, including sensor development, actuator selection, control implementation, and system integration. It also evaluates the internal and external performance of the balance, highlighting accuracy, repeatability, response time, and practical usability. Finally, limitations and potential improvements are discussed to guide future development.

1.0 Overall Design and Methodology

The design of the analytical balance consists of three main stages: the sensor, the actuator, and the control loop that allows auto-balancing. The sensor that detects the balance's balance point consists of an IR emitter, two photodiodes, and a mirror mounted on the rotating beam. A stepper motor attached to a linear screw moves a counterweight along the beam to the balance point. The overall control loop is programmed on an Arduino Nano ESP32 microcontroller, which receives the sensor's output after analog processing and drives a stepper motor to balance the weight at the balance point precisely.

1.1 Sensor Design

To find the exact balance point of the beam, a mirror is attached to the beam 10mm directly above the infrared (IR) emitter at the beam's balance point. This is seen in Figure 1. One photodiode is placed adjacent to the emitter on either side, as shown in Figure 2. As the beam shifts, more IR light is reflected into one of the diodes (See Figure 3), and the difference in light received by the

diodes is converted to a single voltage output, which is input to an analog-to-digital converter (ADC) channel on the Arduino.

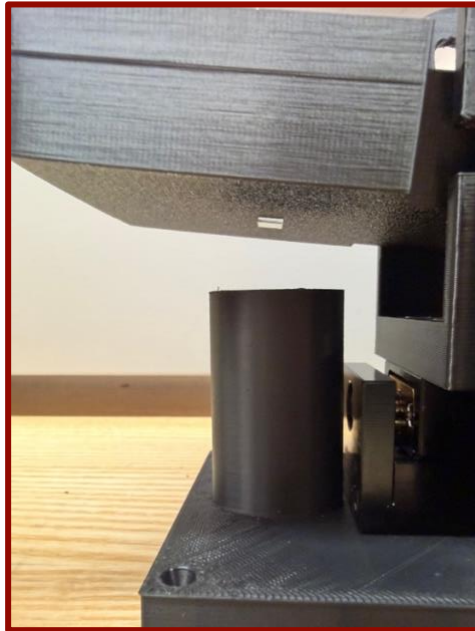


Figure 1: Side view of sensor mounting; sensors placed in a tube to be positioned optimally from the balance beam; the mirror can be seen attached to the beam.

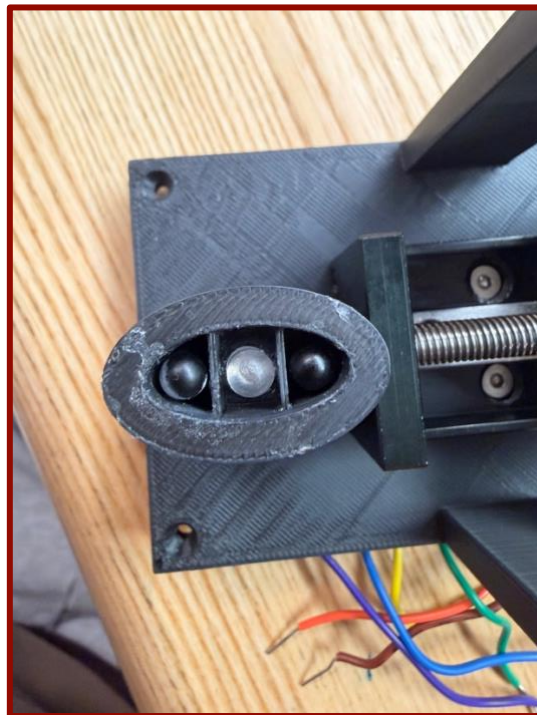


Figure 2: Top view of the sensor, each photodiode is positioned an equal distance from the IR emitter. Walls block any IR leakage.

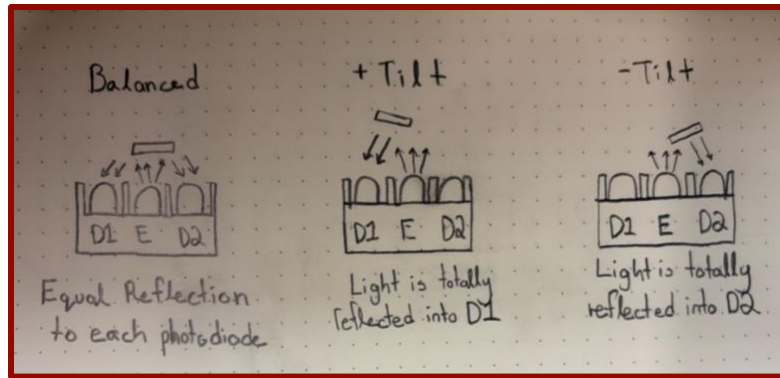


Figure 3: Functional diagram of how the sensor operates as the beam shifts its balance.

1.1.1 Sensor Conditioning

For analog conditioning of the sensor, a transimpedance amplifier (TIA) was used for each photodiode in the first stage. This method of amplification was chosen because the photodiode current changes as it receives more IR light, so it is necessary to convert that photocurrent into a voltage that is measurable by the second stage. As shown in Figure 4, the anode of the photodiode is connected to the 3.3V positive supply, and the cathode is connected to the inverting input of an MCP6002 op-amp. The non-inverting input of this op-amp was connected to a voltage divider that provided half the source voltage, 1.65V. In the feedback loop, a 150kΩ was used to create a transimpedance gain of 150,000 V/A, along with a 22pF capacitor to provide stability. The final output equation of each TIA is $V_{OUT} = 1.65V - (I_{Photodiode} \times 150k\Omega)$. As shown in this equation, as more IR light is received, a larger voltage is subtracted from the 1.65V reference, producing a measurable value for the second stage of the conditioning: an instrumentation amplifier (INA).

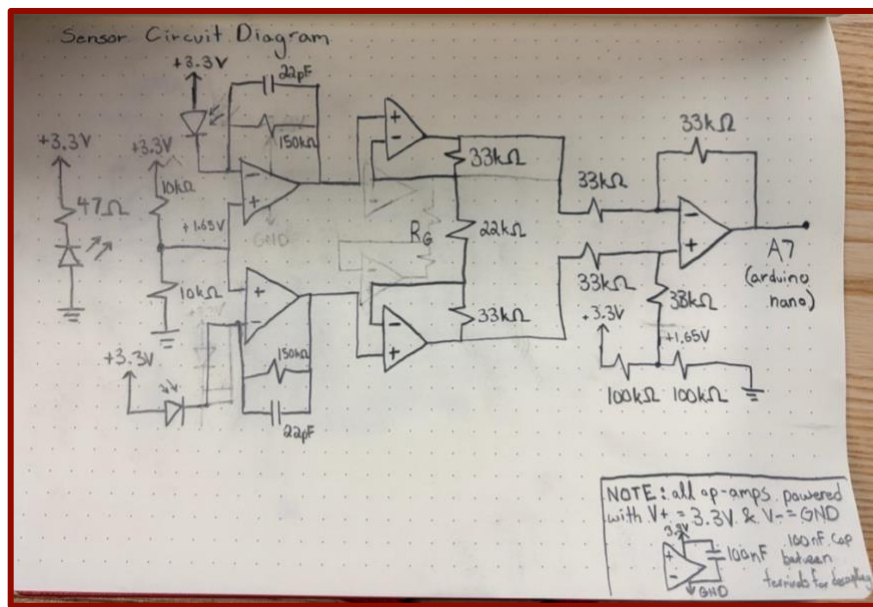


Figure 4: Circuit diagram of the signal conditioning of the sensor reading.

A three-op-amp INA was created using MCP6002 integrated circuits (IC), as seen in Figure 4. This allows the TIA outputs to be compared using the final output equation $V_{OUT} = Gain \times (V_{TIA1} - V_{TIA2}) + V_{REF}$. The reference voltage is +1.65V and $Gain = (2R \div R_G) + 1$ is set by $R = 33k\Omega$, and $R_G = 22k\Omega$, giving a gain value of 4. These were the initial values chosen for sensor amplification, but during testing, the gains of both the TIA and the INA were adjusted to provide a smoother output. The final circuit configuration is shown in the sensor validation section. This direct comparison using a reference voltage in the middle of the Arduino ADC channel range allows the conditioning output to range from 0 to 3.3V, detecting an off-balance of the beam in either direction, with the balance point being 1.65V, when both sensors are receiving an equal amount of reflected IR light, and the difference between them is zero.

1.1.2 Sensor Validation

The values shown in Figure 4 were the initially chosen values that landed on the conservative side to ensure a large output change could be measured throughout the beam's travel. The sensor was then tested to fine-tune the gain and provide a smooth curve as the beam approached the balance point. It was paramount that the sensor could easily detect small changes in the beam's position and provide a wide range of voltage across the change in reflected light.

The first test measured the voltage at both TIA outputs over the beam's travel. The results, shown in Table 1, allowed the gain to be chosen for the INA to provide a detectable distance throughout the beam's travel, enabling precise control by the Arduino. One mirror was used for the first test. Table 2 shows the results from two mirrors to test further smoothing of the photodiode measurements. In both tests, a 150k Ω resistor was used in the feedback loop.

Table 1: TIA voltage output readings, one mirror attached to the beam, directly above the IR emitter.

% Beam Travel (Left to Right)	TIA 1 (Left Photodiode) (V)	TIA 2 (Right Photodiode) (V)
0	1.65	1.00
25	1.65	0.7
50	1.4	1.4
75	0.08	1.55
100	0.50	1.65

Table 2: TIA voltage output readings, two mirrors attached to the beam longitudinally, centered on the IR emitter.

% Beam Travel (Left to Right)	TIA 1 (Left Photodiode) (V)	TIA 2 (Right Photodiode) (V)
0	1.65	1.10
25	0.08	0.70
50	0.08	0.08
75	0.08	1.55
100	0.50	1.65

After the test, a one mirror solution was chosen, because a bigger mirror causes the dead zone of the sensor to be too large, greatly impacting accuracy, as for a large part of the beams travel, both sensors read 0.08 V. With one mirror selected, a lower gain of $100\text{k}\Omega$ was also used to keep the accurate balance point, but reduce the amount that the voltage shifts as the beam rotates, to reduce jitter. Table 3 shows the results using a lower feedback resistor.

Table 3: TIA output voltages using a $100\text{k}\Omega$ feedback resistor.

% Beam Travel (Left to Right)	TIA 1 (Left Photodiode) (V)	TIA 2 (Right Photodiode) (V)
0	1.65	1.20
25	1.65	1.0
50	1.4	1.4
75	0.5	1.65
100	0.8	1.65

Moving on to the INA stage, the output was measured throughout the beam's travel, starting with a gain of 4 (See Table 4). It can be seen with a gain of 4; this is way too high to notice any gradual changes as the beam shifts. There is a very tiny window where the beam balances, and on either edge of this band, the output goes directly to the positive or negative rail of the op-amp. Shifting the gain to 2 using a $68\text{k}\Omega$ yields results shown in Table 5. While this improves the travel recognition, the gain is still too large to see a discernible difference as the beams travel at different points. Finally, moving to a gain of 1 by removing the gain resistor entirely gives the results in Table 6. Here, the output spans the entire beam range, enabling precise actuator control. The final circuit diagram after testing is shown in Figure 5.

Table 4: INA output throughout the beam's travel, gain = 4 ($R_G = 22\text{k}\Omega$).

% Beam Travel (Left to Right)	INA Output (V)
0	3.28
25	3.28
50	1.65
75	0.04
100	0.04

Table 5: INA output throughout the beam's travel, gain = 2 ($R_G = 67\text{k}\Omega$).

% Beam Travel (Left to Right)	INA Output (V)
0	2.70
25	3.00
50	1.65
75	0.08
100	0.32

Table 6: INA output throughout the beam's travel, gain = 1 (gain resistor removed).

% Beam Travel (Left to Right)	INA Output (V)
0	2.20
25	2.70
50	1.65
75	0.60
100	1.00

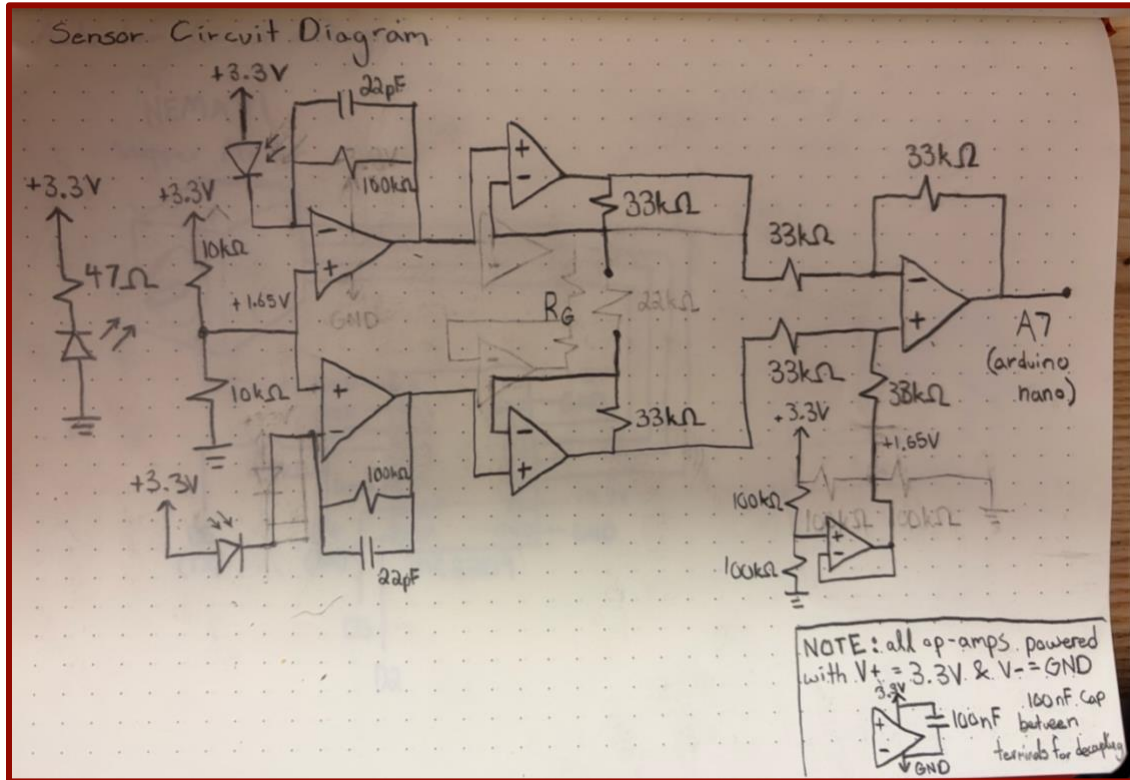


Figure 5: The final circuit configuration after testing.

1.2 Actuator Design

The actuator chosen was a linear screw actuator driven by a NEMA 11 stepper motor. This linear actuator slides a 250-gram counterweight along the beam to balance the beam, shown in Figure 6. The beam itself is mounted on a 4mm-diameter steel pivot rod, connected to the base at the midpoint of the beam using 2 bearings, as seen in Figure 7. The beam's design positions its center of mass as close as possible to the pivot point to remove its mass from the weight calculation.

The counterweight is allowed to move freely in the vertical direction, allowing the beam to rotate with minimal friction for the most accurate measurement. The total travel for the actuator is 100mm, and with a 250-gram counterweight, the moment arm of the weight pan was designed to be 50mm from the pivot point. This allows the counterweight to have more leverage than the weigh pan, satisfying the design constraint of a 50-500g measurement.

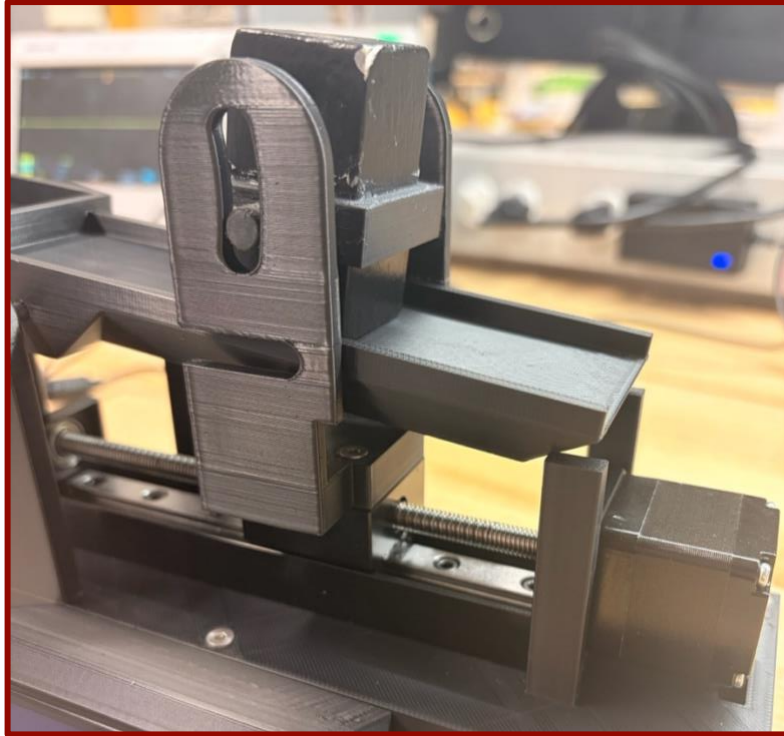


Figure 6: Linear screw actuator with a NEMA 11 stepper motor, attached is the 250-gram counterweight that slides along the beam.



Figure 7: The beam's pivot point, assembled with a 4mm steel rod and 2 bearings attached to the base.

1.2.1 CAD Design

All mechanical parts of the weight scale were designed with respect to the linear actuator and the counterweight. The total mechanical design consists of 4 3D-printed parts, all printed in Polylactic Acid (PLA), chosen for its stiffness and ease of printing [1]. All parts were modeled using OnShape.

The beam consists of a guided platform that allows the weight to slide from the pivot to 100mm from the pivot, as well as a weight pan with its center 50mm from the pivot, measuring 54x54mm. An isometric view of the beam is shown in Figure 8.

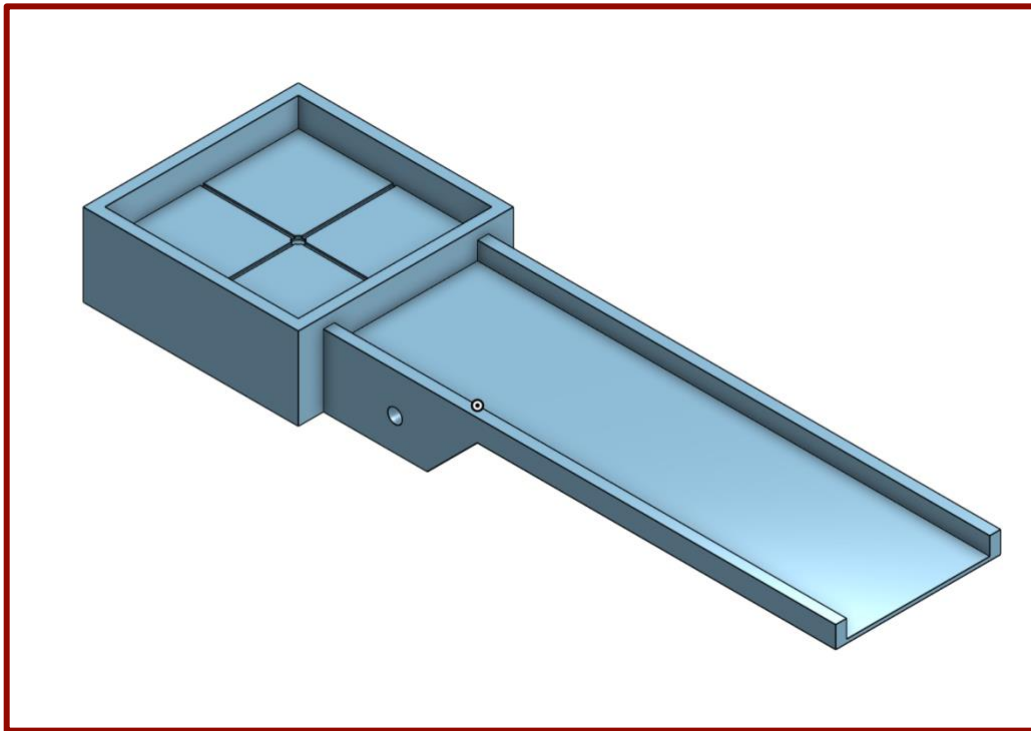


Figure 8: Balance beam that allows the sliding of the counterweight.

The stepper mount screws onto the actuator's sliding plate. It has 2 notches on either side of the beam to allow free vertical movement of the counterweight while translating the actuator's horizontal movement to the counterweight. Figure 9 shows the isometric view of the stepper mount.

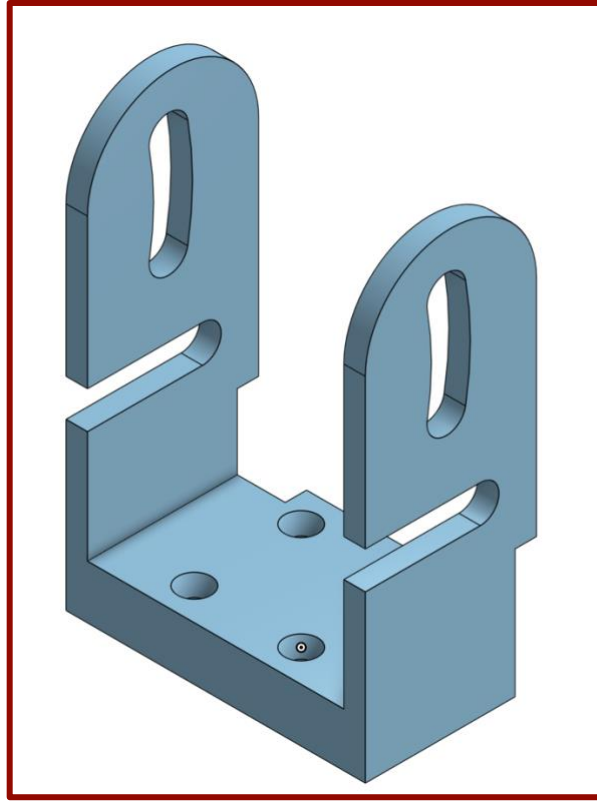


Figure 9: The stepper mount connects to the linear actuator and allows the counterweight to freely move vertically, while horizontal movement is controlled.

The collar is rigidly fixed to the counterweight using glue and has 2 pins that slide along the cutouts on the stepper mount. Figure 10 shows the isometric view of the collar.

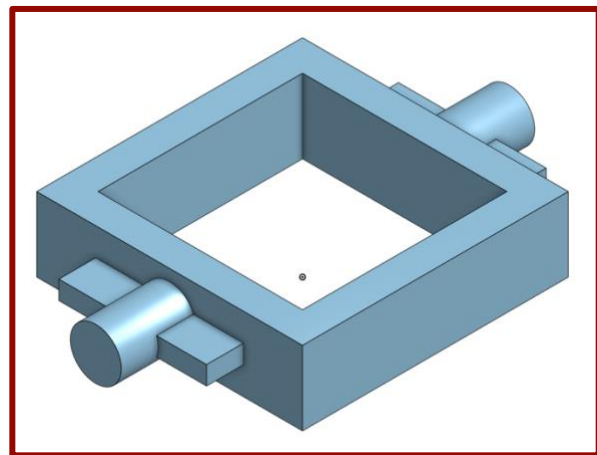


Figure 10: Collar that interacts with the stepper mount.

Finally, the base is what the stepper motor is mounted on, as well as the beam. It has stops at each end to restrict the beam's movement and to position the actuator so that, at full travel, the counterweight is directly above the pivot. Figure 11 shows the base of the balance.

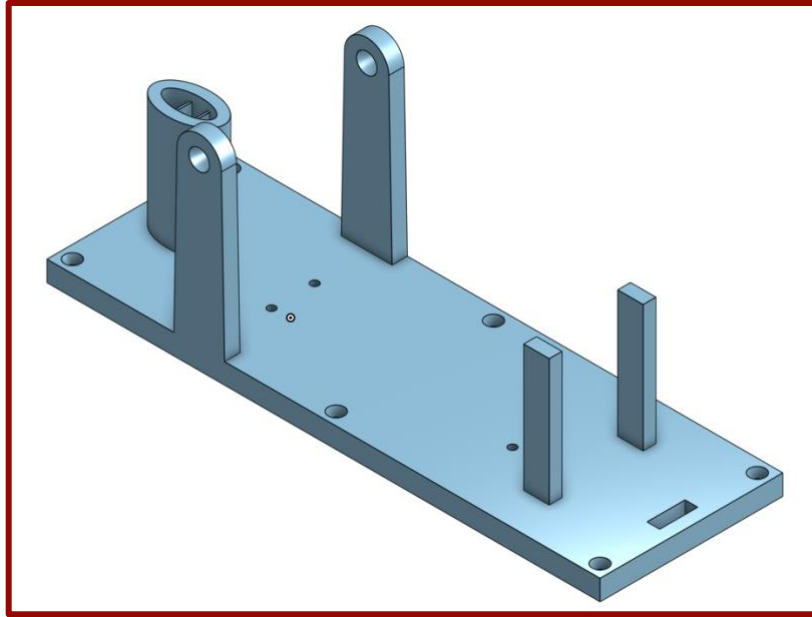


Figure 11: The base of the scale correctly positions the sensor, beam, and actuator for precise measurements.

Once printed, the parts were assembled, and the actuator was mounted on the base, completing the mechanical design, shown in Figure 12.



Figure 12: Full system view, all 3D printed parts assembled to complete the mechanical design.

1.2.2 Controlling the Stepper Motor

The stepper motor is controlled using a TMC2209 stepper motor driver. This driver is commonly used in 3D printers, where stepper motors are widely used. The driver included many features, such as motor stall detection, which was a main reason it was chosen for this project. It also allows micro-stepping up to 1/256 resolution, enabling the NEMA 11 motor chosen, which normally steps 200 times (1.8-degree steps) per cycle, to step 51,200 times per cycle instead, drastically increasing measurement accuracy.

The TMC2209 can be controlled in multiple ways, but with the ESP32, the STEP and DIRECTION pins, as well as the universal asynchronous receiver-transmitter (UART) serial interface, are used to access the driver's full functionality. The step pin is connected to a pulse-width modulation (PWM) signal generated from a general-purpose input-output (GPIO) pin on the ESP32. By varying the frequency of this pulse, the motor speed is controlled. Through UART, microstepping was configured at 1/16 resolution for initial stall detection, then increased to 1/128 during balancing. Figure 13 shows the wiring diagram for the TMC2209, its interface with the motor, and the ESP32.

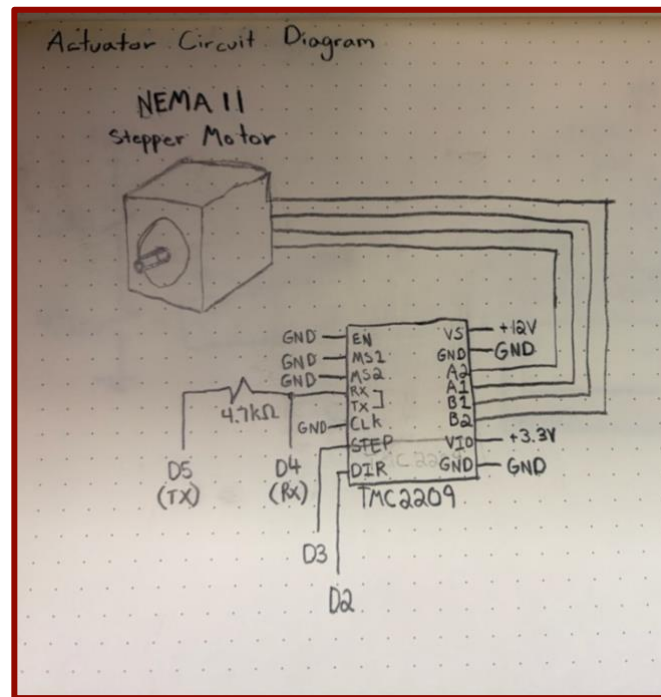


Figure 13: Wiring diagram of the TMC2209 to the NEMA 11 motor and to the ESP32, through the pins D2-D5.

Using the UART interface, we can read a stall value from the driver. This stall value was used to determine when the linear mount hits the end of the sliding screw. By finding this reference each time the balance is powered on, the step count can be reliably set to zero during calibration, allowing repeatable measurements. The motor runs at a much higher speed during this calibration

phase to reduce the balance's overall startup time and to obtain an accurate reading of the stall value over UART, since the motor moves more abruptly from high to low speed during stall.

After calibration, the TMC2209 is commanded over UART to switch to 1/128 microstepping for balance control. This level of resolution over the 100mm screw length allows for 1.28 million unique positions along the range, enabling granular control for the proportional-integral-derivative (PID) loop discussed in the next section.

1.3 Control System Design

There are multiple software-controlled states the balance passes through during operation, as shown in Figure 14.

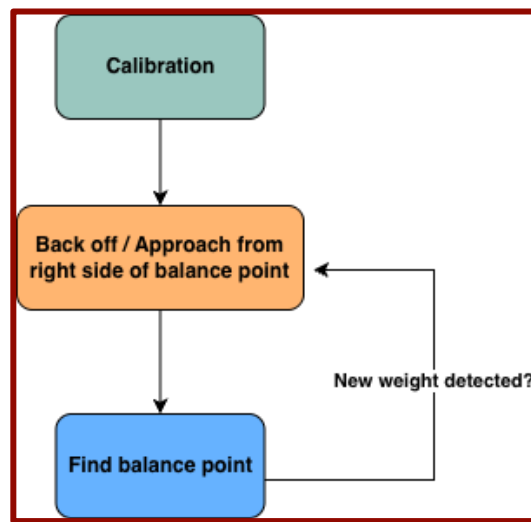


Figure 14: State diagram of the overall program flow.

Upon powering on, the scale immediately begins calibration. In this state, the motor ramps up speed and approaches the linear screw's endpoint, moving away from the motor while checking for a motor stall. Once a stall is detected (meaning the slider has hit the endpoint), the motor will immediately back off from the endpoint (about 500 steps), and the step count is set to zero. This means that the complete bounds for motor travel have now been defined, with a hard limit for the number of steps the motor can take in each direction (0 to 1280000). This software limit ensures the motor does not exceed these bounds, hitting an end stop and potentially damaging the balance.

Once calibration is complete, before approaching the balance point, the ESP32 will read the sensor's voltage input to ensure it is approaching the weight from the motor side (right side) of the balance, that is, when the beam is tilted towards the motor. The sensor's voltage reading is below the 1.65V midpoint. This is done to ensure consistency with all measurements, as there is a small amount of play between the collar and stepper mount. Approaching from the same side each time eliminates any effect this play can have on the measurement.

Finally, once the weight is approaching from the correct direction, the sensor output is polled to determine if it is close to the balance point. Then PID control kicks in, enabling precise motor control to balance the weight at the balance point. In this phase, if a large shift in the voltage reading is detected, the system assumes a new weight has been placed on the scale and returns to the backing-off state to ensure a consistent approach for the new measurement.

1.3.1 Proportional Integral Derivative

From testing, it was observed that as the beam traveled, when it started to move off its end stops on either side, IR light was reflected more directly into the photodiodes, producing a larger voltage difference between them. On each end stop, the voltage reading was 2.2V on the high side and 1.0V on the low side, but as soon as the beam started moving (i.e., close to the balance point), the output increased to 2.9V on the high side and decreased to 0.5V on the low side. This quirk in behavior could be used to move the weight at a constant high speed when the ESP32 knew it was at one of the end stops, and then immediately move into PID control when either a min or max was detected from the sensor, which drastically improves homing speed compared to having PID active all the time.

The balance position is regulated by a PID controller, which drives the stepper motor based on feedback from the photodiode sensor. The sensor outputs a voltage that encodes the mechanical position of the balance beam, and the controller adjusts that voltage to a fixed setpoint. At each sample interval, the controller computes three terms and sums them into a raw output signal:

$$u(t) = K_p \times e(t) + K_i \times \int e(t)dt + K_d \times \frac{de(t)}{dt}$$

where $e(t)$ is the error between the setpoint voltage and the measured sensor voltage, the sign of $u(t)$ is meaningful: a positive output indicates the measured voltage is below the setpoint, commanding the motor to drive in one direction, while a negative output indicates the voltage is above the setpoint, commanding the opposite direction. In this way, the PID output sign directly encodes the required motor direction without any additional logic. The magnitude of the output is then used to determine the motor step frequency according to:

$$f = f_{min} + |u(t)| \times (f_{max} - f_{min})$$

This linearly scales the speed command between the minimum and maximum allowable step frequencies. If the magnitude falls below a minimum command threshold the motor is stopped entirely, and the result is clamped to f_{max} if it would otherwise exceed it. A deadband of ± 0.03 V around the setpoint further suppresses actuation when the balance is sufficiently settled, holding the motor at rest regardless of the computed frequency.

The proportional gain K_p produces a corrective command directly proportional to the current error. A higher K_p value yields a faster response, but if set too high, it causes overshoot and sustained oscillation because the controller overreacts to small deviations.

The integral gain K_i is currently disabled. The integral term accumulates error over time and would eliminate any persistent steady-state offset that proportional action alone cannot correct. Because the system uses an active rehoming routine to handle load changes and because the sensor's response profile makes it difficult to guarantee a stable setpoint without careful logic, it is kept at zero to avoid oscillatory instability.

The derivative gain K_d acts on the rate of change of the error signal, providing anticipatory damping. When the balance is moving quickly toward the setpoint, the derivative term opposes that motion before the error reaches zero, reducing overshoot. To avoid amplifying high-frequency noise, which would cause erratic motor commands, the raw derivative is passed through a low-pass exponential filter with a smoothing factor of $\alpha = 0.12$:

$$d_{filtered}(t) = d_{filtered}(t - 1) + \alpha \times (d_{raw}(t) - d_{filtered}(t - 1))$$

A smaller α retains more of the previous filtered value, producing heavier smoothing at the cost of derivative lag. The value of 0.12 represents a balance between noise rejection and responsive damping.

With $K_i = 0$ and moderate K_d , the dominant oscillation risk comes from the proportional term interacting with mechanical lag. If K_p increased beyond the current value, the system would enter limit-cycle oscillations where the motor continuously hunts around the setpoint without settling. The derivative term counteracts this tendency by braking the approach, and the deadband ensures the motor stops fully once the error is acceptably small, rather than chattering indefinitely. The combination results in a critically-to-overdamped response under normal loading conditions, with rehoming triggered automatically if a sustained large-error condition indicates a new load has been applied. The completed code for this described algorithm can be found in `pid_controller.c` file in the codebase [2].

1.3.2 LCD Output

To allow the user to view the measured weight from the balance, an LCD1602a screen was used to display system statuses and output the weight in grams during measurement. This compact screen was mounted on the base of the balance at a 30-degree angle to allow ergonomic viewing from a sitting or standing position. Figures 15 and 16 show the liquid crystal display (LCD) during the calibration and balance states, and Figure 17 shows the complete balance with the screen powered on.

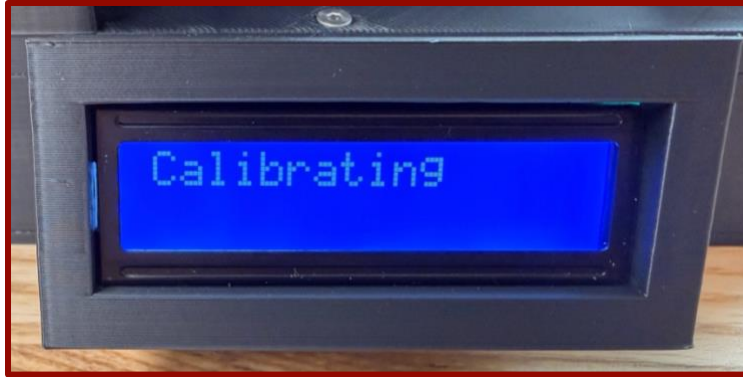


Figure 15: During the calibration state, the LCD outputs "Calibrating".



Figure 16: During balancing, the LCD shows the measured weight in grams, to one decimal point.

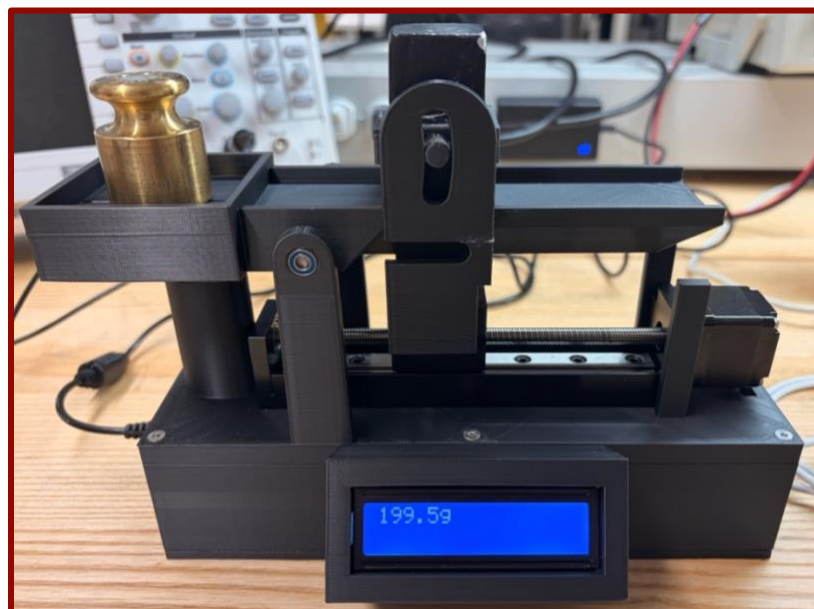


Figure17: The entire powered system; the LCD displays the weight in grams. For reference, a 200-gram scientific weight is being balanced.

2.0 Internal System Performance

2.1 Calibration, Error, Repeatability

2.1.1 Calibrating the Balance

Calibration was done via the comparison method, using scientific calibration weights and recording the number of steps the motor took to balance each weight. Three measurements of each weight were recorded to get the average step value. 50g increments were used from 50g to 400g. Table 7 shows the average values of each weight from the test.

Table 7: Average step value for weights from 50 to 400 in 50-gram increments.

Weight (g)	Average # of Steps (n = 3)
50	189958.33
100	286877.33
150	410964.00
200	520302.67
250	631574.00
300	745263.33
350	866357.67
400	1005252.00

Once these average values were recorded, a piecewise linear, grams-to-steps calculation was used between each 50-gram increment, with linear interpolation to preserve accuracy over the entire range. Table 8 provides the slope (m) and y-intercept (b) for each range of values. For values outside this range, the slope of the minimum and maximum ranges was used for values that go outside of those bounds. Figure 18 displays the line as a plot.

Table 8: Slope and intercept of each section of the piecewise linear calibration function.

Range (g)	Slope (m)	Intercept (b)
50-100	0.000515894716	-47.998498746
100-150	0.000402944168	-15.595547048
150-200	0.000457294752	-37.931680530
200-250	0.000449352048	-33.799070255
250-300	0.000439795010	-27.76093511
300-350	0.000412901214	-7.720122740
350-400	0.000359985897	38.123456875

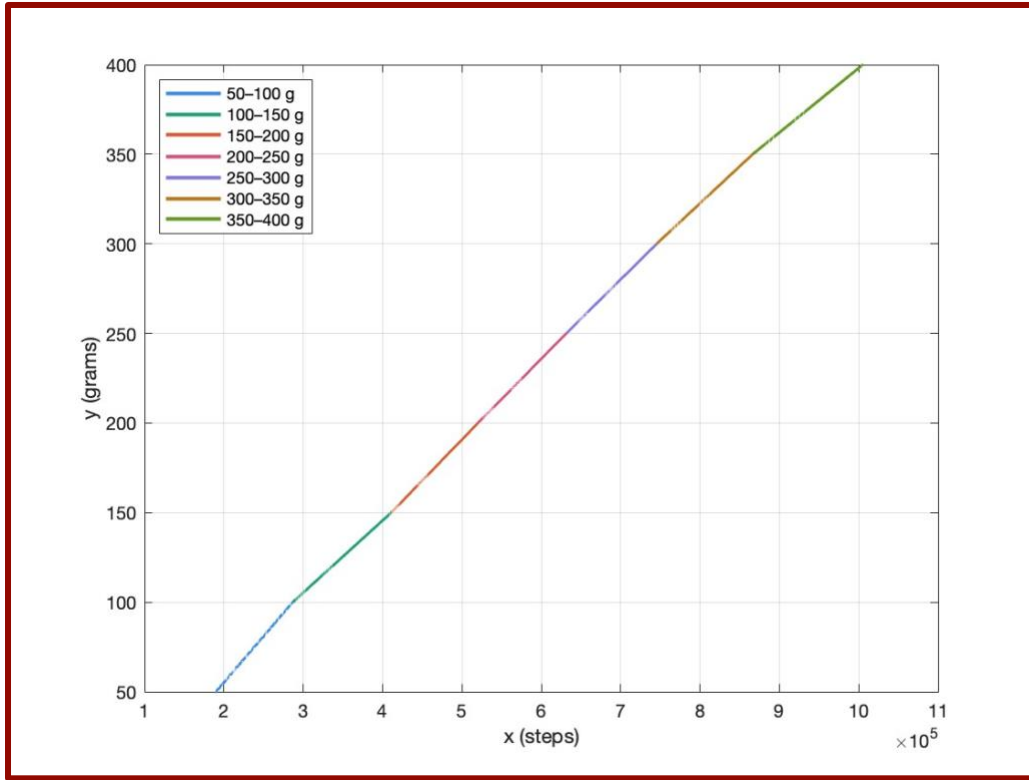


Figure 18: The piecewise function after calibration.

This calibration was implemented in software and was used to display the weight in grams on the LCD screen by converting the step value to a weight.

2.1.2 Error of Measurement

To observe the balance's error and tolerance after calibration, 10 different test weights were used across the balance's range to determine the error relative to the actual weight. Table 9 shows the actual and measured weights across the whole test, as well as the percentage of error.

Table 9: A comparison between the actual weight and measured weight across a range of 10 values.

Actual Weight (g)	Measured Weight (g)	% Error
50	48.7	-2.60%
70	64.5	-7.86%
120	118.7	-1.08%
200	201.9	+0.95%
220	219.0	-0.45%
270	275.8	+2.15%
300	305.3	+1.77%
310	306.4	-1.16%
360	364.1	+1.14%
400	399.3	-0.18%

As shown in Figure 19, throughout the balance's measurement range, it stays extremely close to the actual value, with only one major deviation of -7.86% over the test. From this data, the balance shows reliable accuracy within ± 5 grams of the actual value.

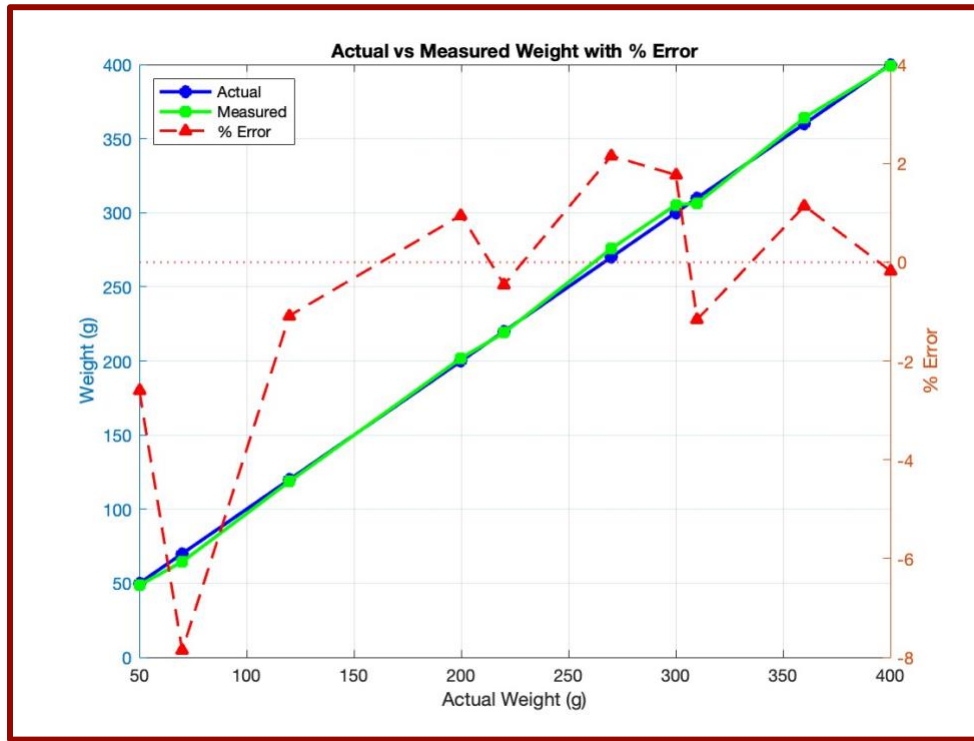


Figure 19: Overlay of both the actual and measured lines, as well as the percentage error of each measurement.

2.1.3 Repeatability of Measurement

To confirm that measurements from the balance were repeatable, a 220g sample was weighed over 10 separate measurements. The process involved placing the weight on the scale, recording the measurement, removing the weight, and repeating. Table 10 shows the results from this test.

Table 10: A 220-gram weight measured 10 times, testing repeatability of measurements.

Measured Weight (g)
223.7
218.6
214.5
225.3
222.3
222.9
221.4
222.6
215.3
219.3

The average across 10 tests was 220.59g, with the largest deviation from this average in each direction being 2.14% and -2.76%.

2.2 Speed of Response

The larger the weight, the farther the counterweight must move from the pivot to balance the beam. A 250-gram and a 400-gram weight were both timed, with the counterweight starting at the pivot, to determine the speed of response when a new weight was placed. The 250-gram weight settled in 15.5 seconds, while the 400-gram weight settled in 19.4 seconds.

Adding additional weights to the pan for small weight adjustments was also tested. Starting at 200 grams and placing a 50-gram weight on top, the balance took 12.23 seconds to settle. Starting with 300 grams and removing 100 grams took 8.71 seconds to settle.

The response speed varies depending on the weight input and whether the counterweight is starting from the pivot point or removing weight from an existing measurement.

2.3 Output Stability

By adjusting the Kp, Ki, and Kd values discussed in 1.3.1, oscillations were eliminated across the entire measurement range, and the motor can hold the counterweight in the exact balance position throughout its travel.

To test the stability of the measurement and drift, a 400-gram and a 200-gram weight were placed on the balance and left for 10 minutes to observe drift over time. Table 11 shows the results of this test, with readings taken every 2 minutes.

Table 11: Stability of measurement across a 10-minute time window.

Actual Weight (g)	Value at each time (2-minute intervals) (g)					
	0:00	2:00	4:00	6:00	8:00	10:00
200	200.7	200.9	201.0	201.0	201.0	201.1
400	401.2	401.3	401.3	401.3	401.3	401.3

As the table shows, the drift across 10 minutes is negligible, highlighting the balance's stability and absence of any oscillation.

3.0 External System Performance

3.1 Sensitivity to Weight Placement and Temperature Effects

The placement of the weight on the scale is quite sensitive; even small errors, such as placing it slightly off-center from the 50mm arm length, affect readings. To test this, a 100-gram weight was placed at different points along the pan, from the far edge to the inner edge. Table 12 shows the

measurements at different points in the pan. To combat this issue, a center mark was extruded in the pan to aid in placing the weight in the desired position (as seen in Figure 20).

Table 12: A test of how placement on the weigh pan affects measurement reading.

Measured weight (100-gram reference)	Distance from center mark (mm)
130.3	+15
121.2	+7.5
102.2	0
80.3	-7.5
73.6	-15



Figure 20: A center mark is on the weigh pan to aid the placement of the weight in the optimal position.

As shown in Table 12, the error can reach 30 grams from a 15mm offset of the weight's center of mass from the beam's center, highlighting the importance of a guide or an auto-positioning design that centers the weight when placed.

As for temperature sensitivity, because an INA is used that compares the difference in output of the photodiodes, and because the photodiodes are positioned about 10mm apart from each other, temperature is a non-issue for the sensor as both diodes will be at relatively the same temperature due to their proximity, and their difference in readings is what is used to detect the balance point.

The stepper motor itself does increase in temperature with prolonged use. This can cause several issues, such as damage to the motor and a loss of magnet strength, reducing motor power and potentially leading to unpredictable behavior (stalls, dropouts) [3]. In use cases lasting up to 1 hour, none of these phenomena were observed, but the motor did get warm to the touch (about 10–15 degrees above ambient).

3.3 Compactness, Ease of Operation, Stopping Mechanism

The main advantage of this balance is its small footprint: the base measures 72 x 218 mm and easily fits on a desk or kitchen countertop for weighing food or as a scientific balance at a workstation.

In terms of ease of use, the user only needs to plug the 12V DC wall power supply into the barrel jack on the balance, as shown in Figure 21. The unit then powers on and self-calibrates, and balancing can begin as soon as the calibration phase completes, within a matter of seconds. To stop using the balance, disconnect the device from power by unplugging it. To allow for quick reprogramming of the balance, a USB port on the other side of the base provides easy access to the ESP32, as shown in Figure 22.

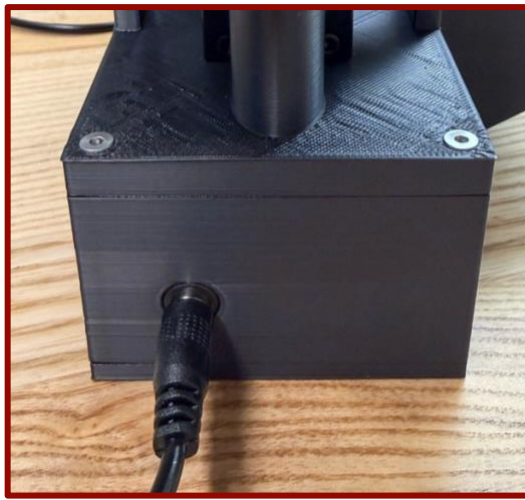


Figure 21: An external DC barrel jack allows simple powering of the balance.



Figure 22: An external USB port on the other side of the base allows for reprogramming of the ESP32.

In terms of device safety, once calibration is complete, the motor cannot move the slider out of the determined bounds, preventing it from hitting either end stop and causing a stall or motor damage.

3.2 Ruggedness and Long-Term Reliability

While the system has low mechanical complexity, the main concern for long-term reliability is wear on the linear screw from repeated movements, as well as bearing wear, even though the bearings are moving at very low speeds. The motor itself does get warm during prolonged use, which is a net negative for long-term reliability and also reduces magnet strength, as previously mentioned. Proper lubrication of the screw, slide rails on the beam, actuator, and bearings extends the lifetime of these parts.

Due to the precise nature of the balance, many small parts with tight tolerances were 3D-printed; specific weak points in terms of ruggedness include the pivot mounts on the base of the scale and the stepper mount. Both of these sections are very thin and are very prone to breaking. During testing, both cracked completely off, and glue was required to refasten them. In a future redesign, adding thickness to both sections would increase durability and reduce the balance's fragility.

As an optical sensor is used, there will be virtually no wear over the scale's lifetime. This is a huge benefit of using optical sensors: there is no mechanical movement, which drastically increases their lifetime compared to other commonly used analytical balance sensors, such as potentiometers.

4.0 Problems and Improvements

4.1 Development and Testing Struggles

The main struggle throughout this project was the limited development window, with the requirement to source many of the parts for the balance externally. Receiving parts on time and then testing them within the required timeframe was difficult.

A major initial issue was making stall detection work on the TMC2209 to allow calibration on startup. After extensive consultation with the datasheet, a custom algorithm was developed for the specific linear actuator used; motor stall could be detected, allowing the balance to be zeroed once powered on.

Another problem was using the LCD1602a with 3.3V logic. The LCD screen is for Arduino use with 5V logic, so it requires 5V to power on and provide sufficient contrast to show visible text. Because the Arduino Nano ESP32 operates at 3.3V logic, it cannot directly power the LCD screen, but it can control it via its GPIO pins. Therefore, a separate 5V supply had to be created from the 12V wall input to power the LCD, while the ESP32 provides control.

Fine-tuning the PID control was another struggle due to a lack of experience with previous control systems. After extensive research, a functioning PID control system was implemented, and testing was conducted to fine-tune and eliminate oscillations.

4.2 Potential Improvements

There are many potential improvements to this balance, namely, restricting the beam's movement will reduce the amount of weight shift once placed on the beam and improve sensor accuracy. Strengthening the pivot connection and the stepper mount would reduce the risk of failure in those components. The pan is also too small to weigh many common objects, whereas this balance has a size advantage; slightly increasing the pan size would greatly improve usability. Reducing friction during the counterweight's sliding and vertical movement would improve balance accuracy.

In terms of electrical design, adding a power switch would improve ease of use by eliminating the need to unplug to power off. A stronger motor would also allow much quicker homing and weight measurement. A custom printed circuit board (PCB) would also reduce the balance's size and improve ruggedness and assembly by using proper connectors for the sensor and actuator.

4.3 Used References

Multiple references were used in completing this project. Namely, Arduino libraries for the TMC2209 [4] and LCD1602a [5] were used as reference points before being ported to ESP32 components for use in the ESP-IDF development environment.

An introduction to PID controller design was used to understand the concepts and implementation of a software-based PID controller [6]. Class learnings, specifically on optical sensors, as well as the textbook, provided a great starting point for implementing the optical sensor and designing the signal conditioning circuit [7].

The complete code for the project is available in a public GitHub repository found here [2].

Conclusion

The automated analytical balance developed in this project successfully demonstrates the integration of optical sensing, precision actuation, and closed-loop control to achieve reliable mass measurement. By combining a photodiode-based infrared sensing system with a stepper motor-driven counterweight and embedded control on an ESP32 platform, the system provides a functional and cost-effective alternative to traditional balance designs.

Performance testing confirmed that the balance achieves acceptable accuracy within ± 5 g across its operating range, with strong repeatability and minimal long-term drift. The implemented proportional-integral-derivative control strategy effectively eliminates oscillations and ensures stable convergence to the balance point. Additionally, the calibration approach using piecewise

linear interpolation provides consistent conversion between actuator position and measured weight.

Despite its successful operation, the system is sensitive to weight distribution and exhibits some mechanical fragility due to 3D-printed components. Response time, while reasonable, could also be improved with enhanced actuator performance. These limitations highlight opportunities for refinement, including mechanical reinforcement, improved user interaction, and further optimization of the control system.

In conclusion, this project validates the feasibility of an optical, actuator-based analytical balance and provides a strong platform for future improvements in accuracy, robustness, and usability.

References

- [1] P. Cambell, "Filament stiffness test: PLA vs the world," V1 Engineering, 1 December 2022. [Online]. Available: <https://forum.v1e.com/t/filament-stiffness-test-pla-vs-the-world/35711>. [Accessed 7 April 2026].
- [2] S. Ruttgaizer, "automated_balance - Github repository," GitHub, 7 April 2026. [Online]. Available: https://github.com/sruttgaizer/automated_balance. [Accessed 7 April 2026].
- [3] R. Bhor and K. Kumar, "Stepper Motor Design Considerations for High-Temperature Applications," Portescap, 22 November 2024. [Online]. Available: <https://www.portescap.com/en/newsroom/blog/2024/11/stepper-motor-design-considerations-for-high-temperature-applications#:~:text=Reduction%20in%20Motor%20Performance:%20As,result%20in%20decreased%20motor%20performance..> [Accessed 7 April 2026].
- [4] P. Polidoro, "TMC2209," Arduino, 9 September 2025. [Online]. Available: <https://docs.arduino.cc/libraries/tmc2209/>. [Accessed 25 March 2026].
- [5] K. Söderby, "Liquid Crystal Displays (LCD) with Arduino," Arduino, 18 11 2021. [Online]. Available: <https://docs.arduino.cc/learn/electronics/lcd-displays/>. [Accessed 30 March 2026].
- [6] Control Tutorials for MATLAB & SIMULINK, "Introduction: PID Controller Design," CTMS, [Online]. Available: <https://ctms.engin.umich.edu/CTMS/index.php?example=Introduction§ion=ControlPID>. [Accessed 27 March 2026].
- [7] J. G. Webster and R. Pallàs-Areny, Sensors and Signal Conditioning 2 Edition, New York: John Wiley & Sons, 2001.